

API INTERVIEW QUESTIONS (100)

Beginner + Advanced + Real-Time Scenarios

SECTION 1: API BASICS (Q1–Q30)

1. What is an API?

Definition: An API (Application Programming Interface) is a set of rules and protocols that allows different software applications to communicate with each other.

Real-time Scenario:

When you use a weather app on your phone, it doesn't have weather data built-in. Instead, it calls a weather service API (like OpenWeatherMap), sends your location, and receives current weather data to display.

2. What is REST API?

Definition: REST (Representational State Transfer) is an architectural style for designing networked applications that use HTTP methods to perform operations on resources.

Real-time Scenario:

A food delivery app uses REST APIs:

- `GET /restaurants` - Fetch available restaurants
- `POST /orders` - Place a new order
- `GET /orders/{id}` - Check order status
- `DELETE /orders/{id}` - Cancel an order

3. What are HTTP methods?

Definition: Standard operations defined by HTTP protocol for web communication.

Real-time Usage:

```
GET /api/products      # Get all products (E-commerce site)
```

```
POST /api/cart          # Add item to cart
PUT /api/users/123      # Update user profile
DELETE /api/comments/5 # Remove comment
PATCH /api/orders/10   # Update order status partially
```

4. Difference between GET and POST?

GET:

- Used to retrieve data
- Parameters visible in URL
- Bookmarkable
- Limited data capacity
- Example: Searching products on Amazon: `amazon.com/search?q=laptop`

POST:

- Used to submit data
- Data in request body (hidden)
- Not bookmarkable
- Larger data capacity
- Example: Submitting login form with username/password

5. What is an endpoint?

Definition: A specific URL where an API can be accessed to perform operations.

Real-time Example:

```
Banking Application:
GET    /api/accounts      → List all accounts
GET    /api/accounts/{id} → Get specific account
POST   /api/transfers     → Initiate money transfer
GET    /api/transactions  → View transaction history
```

6. What is request and response?

Request: What client sends to server

```
POST /api/login HTTP/1.1
Content-Type: application/json
{"username": "john", "password": "pass123"}
```

Response: What server returns to client

```
HTTP/1.1 200 OK
Content-Type: application/json
{"token": "abc123", "user": {"id": 1, "name": "John"}}
```

7. What is JSON?

Definition: JSON (JavaScript Object Notation) is a lightweight data interchange format that's easy for humans to read and write, and easy for machines to parse.

Real-time Scenario:

When you fill out a registration form, your data is converted to JSON:

```
{
  "firstName": "Sarah",
  "lastName": "Chen",
  "email": "sarah@email.com",
  "phone": "+1234567890",
  "address": {
    "street": "123 Main St",
    "city": "San Francisco",
    "zipcode": "94107"
  }
}
```

8. What is HTTP status code?

Definition: A 3-digit code that indicates the result of an HTTP request.

Real-time Examples:

```
200 OK - Your request succeeded (Google search results)
404 Not Found - Page doesn't exist (broken link)
500 Internal Server Error - Website is down
403 Forbidden - You're not allowed to access (admin panel without login)
```

9. Common HTTP status codes?

Success:

- 200 OK - Request successful
- 201 Created - Resource created successfully (new user registered)
- 204 No Content - Success but no data to return

Client Errors:

- 400 Bad Request - Invalid input (wrong email format)

- 401 Unauthorized - Not authenticated (missing token)
- 403 Forbidden - Authenticated but not authorized (user trying to access admin area)
- 404 Not Found - Resource doesn't exist
- 429 Too Many Requests - Rate limit exceeded

Server Errors:

- 500 Internal Server Error - Server problem
- 502 Bad Gateway - Upstream server issue
- 503 Service Unavailable - Server overloaded/maintenance

10. What is RESTful API?

Definition: An API that follows REST architectural principles including statelessness, cacheability, and proper use of HTTP methods.

Real-time Scenario - E-commerce API:

GET	/products	→ Returns list of products
POST	/products	→ Create new product (admin)
GET	/products/{id}	→ Get specific product details
PUT	/products/{id}	→ Update entire product
PATCH	/products/{id}	→ Update product price only
DELETE	/products/{id}	→ Remove product

11. What is statelessness?

Definition: Each request from client to server must contain all the information needed to understand and process the request. Server doesn't store any client context between requests.

Real-time Example:

When you browse products on Amazon:

- Request 1: GET /products
- Request 2: GET /cart (token required)
- Request 3: GET /products/123

12. What is CRUD?

Definition: CRUD stands for Create, Read, Update, Delete.

Real-time Blog System:

```
Create: POST /articles
Read:   GET /articles/{id}
Update: PUT /articles/{id}
Delete: DELETE /articles/{id}
```

13. What is API testing?

Definition: Verifying API functionality, performance, security, and reliability.

Real-time Scenario:

Testing payment APIs for success, failure, load, and edge cases.

14. Tools for API testing?

- Postman
- Swagger/OpenAPI
- SoapUI

15. What is payload?

Definition: Data sent in request or response body.

Example:

```
{
  "content": "Just launched my new app!",
  "privacy": "public"
}
```

Perfect 👍

I'll clearly explain each question (16–30) with:

- Simple definition
- Real-time example
- Interview-friendly explanation
- Practical usage

No shortcuts, no confusion — this is **exactly what interviewers expect**.

16. What are headers?

Definition

Headers are **key–value pairs** sent with an HTTP request or response that contain **metadata** (extra information) about the request or response.

They do **not contain actual data**, but they tell the server **how to process the data**.

Real-time Example

When a user opens a dashboard after login:

```
GET /api/user/profile
Authorization: Bearer eyJhbGciOiJIUzI1Ni...
Content-Type: application/json
Accept: application/json
```

- **Authorization** → Who the user is
- **Content-Type** → Format of request data
- **Accept** → Expected response format

Interview Tip

👉 Headers are mainly used for **authentication, content type, caching, and client information**.

17. What is Content-Type?

Definition

Content-Type header specifies **the format of the data being sent** in the request or response body.

Real-time Examples

1. Sending JSON data

```
Content-Type: application/json
{
  "email": "user@gmail.com",
```

```
  "password": "123456"
}
```

2. File upload

Content-Type: multipart/form-data

Why it is important

If the server receives JSON but Content-Type is missing or wrong, the request may **fail or throw errors**.

Interview Tip

👉 Always set Content-Type correctly, especially for **POST, PUT, PATCH** requests.

18. What is Accept header?

Definition

The Accept header tells the server **which response format the client expects**.

Real-time Example

Mobile App Request

```
GET /api/products
Accept: application/json
```

Server responds:

```
{
  "id": 101,
  "name": "Laptop",
  "price": 50000
}
```

Browser Request

```
Accept: text/html
```

Server responds with an **HTML page**.